Devups

Tome III — L'interface

Documentation du framework

Première édition — novembre 2026

Établie à partir du code réellement exécuté dans le projet **3agReader** (socle Devups 4.3).
Chaque affirmation porte sa source : *fichier:ligne*.

Sommaire

Chapitre 1 — Les annotations de route 5

1.1 Le lecteur d'annotations 5
1.2 Les annotations disponibles 5
1.3 Les segments de chemin 5
1.4 Le piège de la barre oblique 6

Chapitre 2 — Les deux styles d'API 7

2.1 Style générique — <action>/<entité> 7
2.2 Style REST dédié — un FrontController par entité 9
2.3 Style libre — <action>Serve() 11
2.4 Lequel choisir 11

Chapitre 3 — Les contrôleurs 12

3.1 Les quatre familles 12
3.2 Le contrôleur d'administration 12
3.3 CrudTrait — le CRUD en six méthodes 12
3.4 Le remplissage d'une entité depuis une requête 13
3.5 Les écouteurs d'événements 14
3.6 Les contrôleurs de module 14
3.7 La dégradation gracieuse 14

Chapitre 4 — Le moteur de listes 15

4.1 Le contrat déclaratif 15
4.2 Les quatre formes d'une colonne 16
4.3 Les autres clés d'une entrée 17
4.4 Les propriétés de comportement 17
4.5 Les méthodes utiles 18
4.6 Le vocabulaire de la requête 19
4.7 Les opérateurs de filtre 19
4.8 La réponse de lazyloading 20

Chapitre 5 — Le moteur de formulaires 22

5.1 Les trois classes 22
5.2 Le contrat déclaratif 22
5.3 Les types de champ 23
5.4 Les clés d'un champ 23
5.5 Les aides à la construction des options 24
5.6 Les méthodes de FormManager 24
5.7 L'API « à la main » 25
5.8 Un champ traduit 26

Chapitre 6 — Les vues 27

6.1 Blade, via philo/laravel-blade 27

6.2 Genesis — le rendu 27
6.3 La directive @tt 27
6.4 Les fonctions de traduction 28
6.5 Les fonctions globales utiles 28
6.6 Le JavaScript du socle 28

Chapitre 7 — La sécurité 30
7.1 Les deux systèmes d'identité 30
7.2 Le JWT 30
7.3 @Auth, et où il est vérifié 31
7.4 Auth::$restrictions — le modèle réel 31
7.5 Le contrôle de propriété 32
7.6 Le garde-fou des entités d'argent 33
7.7 CORS 34
7.8 Un cas de code mort qui ressemble à de la sécurité 34
7.9 Liste de vérification 34

Ce tome suppose le Tome I lu, en particulier le chapitre 5 (le cycle de vie d'une requête), dont il développe chaque branche.

Chapitre 1 — Les annotations de route

1.1 Le lecteur d'annotations

Devups lit ses annotations avec `maslosoft/addendum`, via `ReflectionAnnotatedClass` et `$method->hasAnnotation('X')` / `getAnnotation('X')`. Ce sont des **annotations en docblock**, pas des attributs PHP 8 :

```
/**
 * @Auth(authorized=1)
 * @GET(path="/:id/comments")
 */
public function comments($id) { ... }
```

Les classes d'annotation vivent dans `dclass/lib/annotation/`.

1.2 Les annotations disponibles

Annotation	Portée	Rôle
`@Api(name='/xxx')`	classe	déclare le préfixe d'URL d'un `FrontController`
`@GET(path='...')`	méthode	route en GET
`@POST(path='...')`	méthode	route en POST
`@PUT(path='...')`	méthode	route en PUT ; `path` vaut `'/:id'` par défaut
`@DELETE(path='...')`	méthode	route en DELETE ; `path` vaut `'/:id'` par défaut
`@Api` (sur une méthode)	méthode	route tous verbes, avec un `path` par verbe
`@Auth(authorized=1)`	méthode	exige une session authentifiée
`@MethodView(GET='...', path='...', Auth=1)`	méthode	l'ancienne forme, utilisée dans `route/Admin.php`
`@DDoc(...)`		documentation
`@OA\...`	classe/méthode	annotations OpenAPI (`zircote/swagger-php`), pour la documentation seulement

`GET`, `POST`, `PUT` et `DELETE` héritent tous de `Api`. Chacun implémente `execute(Router &$route)`, qui apparie l'URL réelle au motif `<name de la classe> . <path de la méthode>` via `Route::parseRoute()`, et verse les paramètres extraits dans `Request::$uri_get_param` et `Request::$uri_get_route_param`. Si l'appariement échoue : `404`.

1.3 Les segments de chemin

Un segment préfixé de `:` est un paramètre nommé :

```
/**
 * @GET(path='/:id')
 */
public function detail($id) { ... }

/**
 * @GET(path='/og/:type/:id')
 */
public function index($type = null, $id = null) { ... }
```

Le nom du paramètre d'URL doit correspondre au **nom du paramètre PHP** : c'est par ce nom que `mapFunction()` et `getMethodParamValues()` lient la valeur.

Un `path` peut porter **plusieurs motifs séparés par des virgules** : `Api::execute()` les essaie l'un après l'autre et retient le premier qui a le bon nombre de segments.

1.4 Le piège de la barre oblique

Mesuré sur 3agReader, le 2026-10-06, sur `GET /api/share/og` :

`$ctrlApi->name` vaut `/share` et le `path` de l'annotation `/og`. La concaténation directe donnait `\share/og` — **barre oblique inversée** — là où l'URI vaut `/share/og`. Aucune annotation de sous-ressource ne pouvait correspondre, et le résultat était un `404` sans explication.

La forme correcte, désormais dans `Request.php:490` :

```
$pattern = rtrim($ctrlApi->name, '/') . '/' . ltrim($ann->path, '/');
```

La barre oblique initiale du `path` est obligatoire. `@GET(path='/og/:type/:id')`, jamais `@GET(path='og/:type/:id')`.

Chapitre 2 — Les deux styles d'API

C'est la distinction à ne pas confondre. Les deux styles sont vivants, sur les mêmes entités, et ne se comportent pas pareil.

2.1 Style générique — `<action>/<entité>`

Piloté par `WebService::systemServe()` et les méthodes `createCore()`, `updateCore()`, `deleteCore()`, `detailCore()`, `uploadCore()`, `ll()`, `dcollection()` de `dclass/devups/Controller/FrontController.php`.

L'entité doit figurer dans `$public_access` de `route/WebService.php`, **et** dans `config/dvups_configurations.php` — `Controller::getclassname()` vérifie les deux et lève sinon :

```
entity <x> has no public access or has not been found
entity <x> not found in $global_config
```

Action	Route	Méthode du socle
Créer	`POST /api/create/<entité>`	`createCore()`
Détail	`GET /api/detail/<entité>?id=X`	`detailCore($id)`
Lister	`GET /api/lazyloading/<entité>?dfilters=on&...`	`ll()`
Mettre à jour	`/api/update/<entité>?id=X`	`updateCore($id)`
Supprimer	`/api/delete/<entité>?id=X`	`deleteCore($id)`
Envoyer un fichier	`/api/upload/<entité>?id=X`	`uploadCore($id)`
Lectures groupées	`POST /api/dcollection`	`dcollection()`

> **Sur les verbes HTTP de ce style.** Le mémo de BGFIBank note que `POST` fonctionnait pour `update` et `GET` pour `delete`, alors que la documentation d'origine montre `PUT` et `DELETE`. Le dispatcher ne contrôle en effet **pas** le verbe sur ce style : `runServe()` teste seulement le premier segment contre la liste des sept actions. Les verbes sont donc tolérants. Ne pas s'en réjouir : écrire le verbe correct reste la bonne pratique, et une version future peut resserrer.

La charge utile

```
{ "<entité>": { "champ": "valeur", "relation.id": 3 } }
```

Le **point** référence une relation `manyToOne`. Et pour une création en masse :

```
{ "<entité>_bulk": [ { ... }, { ... } ] }
```

`createCore()` traite alors chaque élément séparément et renvoie un tableau `datacollection` avec un `success` par ligne.

L'envoi de fichiers

Un attribut de `formtype` binaire (`image`, `file`) ne passe pas en JSON : il faut du `multipart/form-data`, avec des clés `<entité>_form[champ]` au lieu de l'objet `{"<entité>": {...}}`.

```
comicbook_form[name]
comicbook_form[description]
comicbook_form[cover]    (le fichier)
```

`createCore()` détecte alors `$_FILES[<entité>_form]` et enregistre un écouteur d'après-création `upload<Champ>` par fichier.

Deux pièges critiques de `lazyloading`

1. `?<relation>_id=X` ne filtre rien. Vérifié empiriquement sur un autre projet : `GET /lazyloading/entity?project_id=1` renvoyait `nb_element: 1021` — le total de **toute** la table, tous projets confondus — alors que le projet 1 n'avait que 21 entités. Le bon filtre relationnel est :

```
?dfilters=on&project.id:eq=1
```

qui donne bien `nb_element: 21`. Le premier semble marcher — `200`, `success: true`, des résultats reviennent — et renvoie silencieusement tout le monde.

2. La réponse est paginée, silencieusement. Par défaut `per_page = 30` (`Lazyloading.php:25`). La réponse porte `nb_element` (le total réel du filtre), `per_page`, `current_page`, `next`, `remain` (1 tant qu'il reste des pages, 0 sur la dernière). **Les paramètres de pagination sont `?next=<page>&per_page=<taille>`** — pas `?page=`, ni `?current_page=`, ni `?p=`, testés et non fonctionnels.

Tout code qui appelle `lazyloading` sans boucler sur `remain` / `next` affiche une liste incomplète dès que le résultat filtré dépasse 30 lignes :

```
let all = [], page = 1, remain = true
while (remain) {
  const res = await apiClient.get(
    `/lazyloading/xxx?dfilters=on&project.id:eq=${projectId}&next=${page}&per_page=100`
  )
  all = all.concat(res.data.listEntity || [])
  remain = !!res.data.remain
  page = res.data.next
}
```

> Ces deux pièges combinés ont déjà produit une conclusion erronée sur un projet réel (« l'API ne persiste rien ») : les données étaient bien créées et bien associées, mais cherchées dans un résultat non filtré que l'on croyait tronqué.

2.2 Style REST dédié — un `FrontController` par entité

Généré par `php devups core:g:frontcontroller`. La classe porte `@Api(name='/xxx')`, les méthodes portent une annotation de verbe.

```
/**
 * @Api(name='/task')
 */
class TaskFrontController extends FrontController
{
    /**
     * @Auth(authorized=1)
     * @GET
     */
    public function index()
    {
        return Task::initQb()->with('project', ['name'])->lazyloading();
    }

    /**
     * @Auth(authorized=1)
     * @GET(path='/:id')
     */
    public function detail($id) { ... }

    /**
     * @Auth(authorized=1)
     * @POST
     */
    public function create() { ... }

    /**
     * @Auth(authorized=1)
     * @PUT(path='/:id')
     */
    public function update($id) { ... }

    /**
     * @Auth(authorized=1)
     * @DELETE(path='/:id')
     */
    public function delete($id) { ... }
}
```

Méthode PHP	Annotation	Route
`index()`	`@GET`	`GET /api/task`
`create()`	`@POST`	`POST /api/task`
`detail($id)`	`@GET(path='/:id')`	`GET /api/task/42`
`update($id)`	`@PUT(path='/:id')`	`PUT /api/task/42`
`delete($id)`	`@DELETE(path='/:id')`	`DELETE /api/task/42`

Ce style est strict sur le verbe HTTP. Une méthode annotée `@GET` appelée en POST répond `405`.

Le dispatch, mesuré

`Request::FrontController($route, $path, $ctrl)` (`Request.php:445`) :

1. `niceFunction($path, '-')` convertit le segment d'action en nom de méthode : `confirm-order-with-wallet` → `confirmOrderWithWallet`.
2. **Contrôle de cohérence** : `"/" . Request::$uri_get_param["dclass"]` doit être égal au `name` de l'annotation `@Api` de la classe, sinon `500` — « Controller name X no match with entity name ».
3. **Si la méthode n'existe pas** : balayage de toutes les méthodes publiques qui portent une annotation du verbe courant avec un `path` non vide, et appariement du motif complet à l'URI. **C'est le mécanisme de sous-ressource** (`/api/share/og/:type/:id`).
4. **Si rien n'a été apparié** : résolution par **nombre de segments** :

Segments	Verbe	Méthode
1 (`/task`)	GET / POST / PUT / DELETE	`index` / `create` / `update` / `delete` (via `Router::$default_path`)
2 (`/task/42`)	GET	`detail`
2	PUT / DELETE	`update` / `delete`
2	POST	**`404`** — « Request method POST not found with 2 paths »
3+	tous	**`404`** — « Request method X not allowed for this route »

1. **Contrôle d'habilitation** contre `Auth::$restrictions`, sur le nom de méthode **réellement résolu**.
2. **`@Auth`** sur la méthode : `Auth::execute()`.
3. **Le verrou d'annotation** : si la méthode n'a ni annotation du verbe courant, ni `@Api` → `405` (`Request.php:577-585`).
4. Appel, avec les paramètres liés par nom.

> **Divergence avec le mémo de BGFIBank, à signaler.** Le mémo affirme qu'un `@GET(path='/hello')` sur une méthode `hello()` « n'est jamais lu » et que le dispatch ne route que sur le nombre de segments. **Ce n'était vrai que d'une version antérieure du socle.** Le code de 3agReader contient bien l'étape 3 ci-dessus, et elle fonctionne : `GET /api/share/og/{type}/{id}` est une route de sous-ressource réelle, en production. Le contournement que décrit le mémo — exposer l'action comme un `xxxServe()` sur `route/WebService.php` — reste valable, mais il n'est plus nécessaire.
>
> **Conséquence de l'étape 7, vérifiée sur 3agReader.** Deux méthodes publiques de `OrdersFrontController` — `confirmOrderWithWallet()` et `subscribeAction()` — n'ont **aucune** annotation. `niceFunction()` résout bien leur nom, donc `method_exists()` est vrai et le balayage de l'étape 3 est sauté ; l'étape 7 répond alors `405` pour tous les verbes. **Ces deux méthodes ne s'exécutent jamais, par aucune route.** Une méthode nommée, publique, qui ressemble à un chemin de paiement, et qui est en réalité morte : c'est exactement le genre de chose que ce verrou rend invisible. **Vérifier l'annotation avant de conclure qu'une méthode de `FrontController` est appelée.**

2.3 Style libre — `<action>Serve()`

Troisième voie, pour tout ce qui n'est ni une entité ni un verbe générique : une méthode `<Action>Serve()` directement sur `route/WebService.php`.

```
// GET /api/hello  →  helloServe()
public function helloServe() { ... }

// POST /api/accept-invitation?id=5  →  acceptInvitationServe()
public function acceptInvitationServe() { ... }
```

Les paramètres passent en chaîne de requête, pas en segment de chemin. Sur 3agReader : `helloServe`, `cacheServe`, `testRatingServe`, `homeGenderServe`, `contentServe($ref)`, `shareServe($path, $id)`, `systemServe`.

C'est la bonne porte pour une action métier qui ne se range pas dans le CRUD d'une entité. Attention : **`mapFunction()` n'exige pas d'annotation ici** (Tome I §5.7), donc `@Auth` est à poser explicitement, ou le contrôle à écrire à la main :

```
if (!Auth::$user_id) {
    http_response_code(401);
    Genesis::json_encode(['success' => false, 'detail' => 'non authentifié']);
}
```

`Auth::$user_id` est déjà peuplé à ce stade : `runServe()` appelle `Auth::execute()` en tête, une fois pour toutes.

2.4 Lequel choisir

Besoin	Style
CRUD immédiat d'une entité d'administration, sans écrire de contrôleur	**générique**
API consommée par une application mobile ou un front séparé, avec un contrat stable	**REST dédié**
action métier qui n'est pas un CRUD	**libre** (`xxxServe`)
sous-ressource (`/entité/:id/commentaires`)	**REST dédié**, avec `@GET(path='/:id/commentaires')`

Le style générique est pratique et dangereux : il expose les cinq actions d'écriture de toute entité enregistrée. Le chapitre 7 décrit les garde-fous qui ont dû être posés autour.

Chapitre 3 — Les contrôleurs

3.1 Les quatre familles

Classe	Rôle	Dispatch
`<Entité>Controller`	le CRUD du back-office	`Request::Controller()`
`<Entité>FrontController`	l'API REST dédiée	`Request::FrontController()`
`Module<Nom>` (extends `ModuleController`)	les écrans d'un module	`Request::Module()`
`route/App`, `route/WebService`, `route/Admin`, `route/AdminService`	les quatre portes	`Router`

3.2 Le contrôleur d'administration

`Request::Controller()` (`Request.php:389`) convertit le **dernier segment** de l'URL en nom de méthode :

```
$function = Util::toPascalCase($path);
if (in_array($function, ["create","update","delete","_delete","deletegroup","deleteGroup"]))
    $function .= "Action";
else if (in_array($function, ["form","detail","list"]))
    $function .= "View";
```

URL	Méthode
`/admin/mp/ModuleTask/task/list`	`listView()`
`/admin/mp/ModuleTask/task/detail?id=4`	`detailView(4)`
`/admin/mp/ModuleTask/task/form?id=4`	`formView(4)`
`.../task/create`	`createAction()`
`.../task/update?id=4`	`updateAction(4)`
`.../task/deletegroup`	`deletegroupAction()`

3.3 `CrudTrait` — le CRUD en six méthodes

Un `<Entité>Controller` généré utilise `CrudTrait` (`dclass/devups/Controller/CrudTrait.php`), qui fournit :

Méthode	Rôle
`listView($next, $per_page)`	construit la liste par `<Entité>Table::init(...)->buildindextable()`, charge le JS de l'entité, rend la vue

Méthode	Rôle
`datatable($next, $per_page)`	renvoie la liste en JSON, pour le rechargement sans rechargement de page
`formView($id = null)`	construit le formulaire par `<Entité>Form::init($entity, $action)->buildForm()->addDformjs()->renderForm()`
`createAction()` / `updateAction($id)`	remplissent l'entité par `form_fillingentity()`, renvoient les erreurs ou la ligne de tableau mise à jour
`detailView($id)`	rend la vue de détail par `builddetailtable()->renderentitydata($entity)`
`deleteAction($id)` / `deletegroupAction($ids)`	suppression simple et en masse
`_editAction($id)` / `_newAction()`	alias de `formView()`

Le contrôleur généré se contente de déclarer les quatre noms statiques :

```
class TaskController extends Controller
{
    use CrudTrait;

    public static $classname  = 'Task';
    public static $entityname = 'task';
    public static $tablename  = 'TaskTable';
    public static $formname   = 'TaskForm';
}
```

> **`updateAction()` a le défaut `dvold`** (Tome II §2.4) : il appelle `new self::$classname($id)`, pas `find($id)`. Un formulaire d'administration qui enregistre sans rien écrire a probablement cette cause.

3.4 Le remplissage d'une entité depuis une requête

Trois méthodes de `Controller` font ce travail, et elles ne sont pas interchangeables.

Méthode	Entrée	Usage
`form_fillingentity($object, $data)`	`$_POST[<entité>_form]`	les formulaires du back-office
`hydrateWithJson($object, $jsonform, $deeper = false)`	un tableau décodé depuis le corps JSON	l'API
`form_generat($object, $data, $deeper)`		

Les deux premières alimentent `$this->error`, un tableau indexé par nom de champ. Un contrôleur vérifie `if ($this->error)` avant de persister, et renvoie le tableau au client.

`formWithPost()` (privée, `Controller.php:298`) est le cœur du remplissage : pour chaque champ, elle cherche un `set<Champ>()` sur l'entité et, s'il existe, l'appelle en traitant **sa valeur de retour comme un message d'erreur** (Tome II §5.4).

3.5 Les écouteurs d'événements

```
Controller::addEventListener($position, $action, $classname);
Controller::addEventListenerAfterCreate($classname, $method);
```

`createCore()` s'en sert pour enchaîner un `upload<Champ>()` après la création, quand des fichiers accompagnent la requête.

3.6 Les contrôleurs de module

Un `Module<Nom>.php` hérite de `ModuleController`, donc de `Router`. `Request::Module()` (`Request.php:266`) convertit le segment en `<segment>View` et instancie la classe avec `'layout'` en argument.

> **Deux pièges mesurés.**
>
> Le constructeur d'un `Module<X>` attend un `$default_url` (une chaîne), jamais l'objet `Router` — malgré un nom de paramètre `$route` identique dans les signatures générées. `Request::Module()` passe donc littéralement `'layout'`.
>
> La barre latérale du back-office génère un lien `admin/<projet>/<module>/dashboard` pour **tous** les modules, mais la plupart n'exposent que `layoutView()`. `ModuleController::dashboardView()` est donc un repli : il appelle `layoutView()` si elle existe, et meurt proprement sinon. Les modules qui définissent leur propre `dashboardView()` l'emportent par la résolution de méthode ordinaire de PHP.

3.7 La dégradation gracieuse

Un écran dont le code n'a jamais été généré ne doit pas produire une erreur fatale nue. `Request::Controller()` et `Request::Module()` traitent ce cas (`Request.php:406`, `:311`) :

- si la classe de contrôleur n'existe pas du tout, ou
- si une `Throwable` est une erreur « Class … not found »,

alors `renderNotMonitored()` rend `dclass/devups/Resource/views/not_monitored.blade.php` — un écran « contenu non monitoré ».

Seule une erreur de classe introuvable est dégradée. Un vrai bug — une méthode existante mais cassée, un mauvais type d'argument — continue de planter bruyamment : le message serait un mensonge sinon.

Chapitre 4 — Le moteur de listes

4.1 Le contrat déclaratif

Une liste du back-office **ne s'écrit pas en HTML**. Elle se déclare dans une classe `<Entité>Table` qui hérite de `Datatable`, et un moteur unique (`dclass/devups/Datatable/Datatable.php`, 55 ko) rend toutes les listes du projet.

```
class TaskTable extends Datatable
{
    public $entity = "task";

    public static function init(\Task $task = null)
    {
        $dt = new TaskTable($task);
        $dt->entity = $task;
        return $dt;
    }

    public function buildindextable()
    {
        $this->order_by = "this.id desc";
        $this->datatablemodel = [
            ['header' => t('task.id', '#'),      'value' => 'id'],
            ['header' => t('task.name', 'Nom'),  'value' => 'name'],
            ['header' => t('entity.project'),    'value' => 'Project.name'],
            ['header' => t('task.done'),         'value' => function (\Task $t) {
                return $t->done ? '<span class="badge">fait</span>' : '';
            }],
        ];
        return $this;
    }

    public function builddetailtable()
    {
        $this->datatablemodel = [
            ['label' => t('task.name'), 'value' => 'name'],
        ];
        return $this;
    }

    public function router()
    {
        $tablemodel = Request::get("tablemodel", null);
        if ($tablemodel && method_exists($this, "build" . $tablemodel . "table")
            && $result = call_user_func([$this, "build" . $tablemodel . "table"])) {
            return $result;
        }
        return $this->buildindextable();
    }
}
```

4.2 Les quatre formes d'une colonne

C'est le point le plus important du chapitre, et le moins évident. `Datatable::tablebodybuilder()` boucle :

```
foreach ($this->datatablemodel as $attrib => $valuetd) { ... }
```

`$attrib` est la **clé** de l'entrée, `$valuetd` sa valeur. Quatre formes sont reconnues, dans cet ordre :

Forme 1 — `'get'` appelable. Si l'entrée porte `'get'` et que c'est un appelable, il est appelé avec l'entité, et avec `'param'` en second argument s'il est présent :

```
['header' => 'Couverture', 'get' => function (\Task $t, $param) { ... }, 'param' => [...]]
```

Forme 2 — pas de `'value'` du tout : la clé EST la colonne.

```
'name'       => ['header' => t('Nom')],          // → $entity->name
'project.id' => ['header' => t('Projet')],        // → $entity->project->id
```

C'est la forme que le générateur produit, et c'est la convention normale du framework : la clé nomme la colonne, l'entrée ne porte que l'en-tête et les options. Un nom pointé devient `$entity->{a}->{b}`.

> **Conséquence, et défaut à connaître.** Si la clé ne correspond à aucune propriété réelle, l'accès `$entity->{$attrib}` échoue dans `ModelTrait::__get()` — qui, sur un attribut non déclaré, meurt par `die()` — et **l'écran entier ne s'affiche pas**. Sur 3agReader, une seule clé mal orthographiée (`'paymentref'` au lieu de `'discountprice'`, dans une entrée dont l'en-tête disait pourtant « Discountprice ») suffisait à casser la liste des commandes. L'en-tête et la clé doivent se correspondre, et la clé doit nommer une colonne réelle.

Forme 3 — `'value'` appelable.

```
['header' => 'Statut', 'value' => function (\Task $t) { return $t->done ? 'oui' : 'non'; }]
```

C'est la forme à utiliser pour tout ce qui demande un calcul, un formatage HTML, ou une lecture indirecte. Elle reçoit l'entité hydratée.

Forme 4 — `'value'` chaîne.

```
['header' => 'Nom',    'value' => 'name']          // → $entity->name
['header' => 'Projet', 'value' => 'Project.name']   // → $entity->getProject()->getName()
['header' => 'Statut', 'value' => 'status.colortab']
```

Une chaîne pointée est résolue par **accesseurs** (`get<A>()` puis `get<B>()`), pas par accès direct. D'où la nécessité que les accesseurs existent — c'est la cause la plus fréquente des listes cassées dans un back-office Devups : `class User does not have a method "getUsername"`.

Deux préfixes spéciaux sont reconnus dans une chaîne de valeur :

Préfixe	Effet
`src:<champ>`	appelle `show<Champ>()` — pour un fichier ou une image
`<Relation>::<collection>`	parcourt une collection

4.3 Les autres clés d'une entrée

Relevé exhaustif des clés que `Datatable.php` lit sur une entrée :

Clé	Rôle
`header`	l'en-tête, pour une liste (`buildindextable`)
`label`	l'en-tête, pour un détail (`builddetailtable`)
`value`	la colonne, l'appelable, ou la chaîne pointée
`get`	un appelable prioritaire sur `value`
`param`	le second argument passé à `get`
`search`	active ou désactive le champ de filtre de cette colonne
`order`	contrôle le tri sur cette colonne
`sql`	une expression SQL ajoutée à la projection par `addColumn()`, aliasée sur la clé de l'entrée
`directive`	des attributs HTML supplémentaires

L'entrée `'sql'` est le mécanisme pour afficher une colonne calculée :

```
'total' => [
    'header' => t('Total'),
    'sql'    => '(SELECT SUM(amount) FROM orders_item WHERE orders_id = this.id)',
],
```

`getTableRest()` (`Datatable.php:716-720`) parcourt le modèle avant d'exécuter la requête et appelle `$this->qbcustom->addColumn($value['sql'], $key)` pour chaque entrée qui en porte une.

4.4 Les propriétés de comportement

Propriété	Défaut	Rôle
`$datatablemodel`	`[]`	**le contrat** (§4.2)
`$order_by`	`""`	le tri par défaut, ex. `"this.id desc"`
`$base_url`	`""`	la racine des liens d'action
`$per_page`	`10`	la taille de page
`$pagejump`	`10`	le saut de pagination
`$enablepagination`	`true`	
`$per_pageEnabled`	`true`	

Propriété	Défaut	Rôle
`$searchaction`	`true`	la ligne de filtres
`$groupaction`	`true`	les cases à cocher de sélection multiple
`$group_field_name`	`"id"`	le nom du champ de sélection
`$isRadio`	`false`	sélection unique au lieu de multiple
`$enabletopaction`	`true`	la barre d'actions du haut
`$defaulttopaction`	`true`	le bouton « ajouter » par défaut
`$enablecolumnaction`	`true`	la colonne d'actions
`$actionDropdown`	`true`	les actions en menu déroulant
`$defaultaction`		
`$mainrowaction`	`"edit"`	l'action principale d'une ligne
`$customactions`	`[]`	des actions supplémentaires par ligne
`$topactions` / `$groupactions` / `$rowaction`	`[]`	
`$additionnalrow`	`[]`	des lignes de pied de tableau
`$template`	`""`	un gabarit de rendu personnalisé, au lieu du tableau
`$viewdata`	`[]`	des données passées à ce gabarit
`$noidentity`	`false`	
`$isFrontEnd`	`false`	
`$trattribut_callback`	`null`	un appelable qui pose des attributs HTML sur la ligne
`$qbcustom`	`null`	un `QueryBuilder` personnalisé

4.5 Les méthodes utiles

Méthode	Rôle
`init($entity)`	le constructeur conventionnel
`buildindextable()`	**à implémenter** : le modèle de la liste
`builddetailtable()`	**à implémenter** : le modèle de la fiche de détail
`build<X>table()`	un modèle alternatif, sélectionné par `?tablemodel=X`
`router()`	choisit le modèle selon `?tablemodel=`
`render()`	la page complète (HTML)
`getTableRest($datatablemodel = [])`	la liste en JSON : `tablebody`, `tablehead`, `tablepagination`
`getSingleRowRest($entity)`	une seule ligne, pour une mise à jour sans rechargement
`renderentitydata($entity, $callback)`	le rendu d'une fiche de détail

Méthode	Rôle
`Qb($callback)`	injecte un `QueryBuilder` personnalisé
`addFilterParam($param, $value)`	ajoute un filtre figé
`setModel($name)`	sélectionne un modèle
`addcustomaction($action)` / `addgroupaction($action)`	
`setDefaultAction($action, $callback)`	
`crud_url($read, $update, $delete)`	surcharge les URLs d'action
`disablepagination()` / `setperpage($n)` / `setjumppage($n)`	
`enablefilter($bool)` / `disableColumnAction()` / `disableDefaultaction()`	
`exportCsv()`	export, déclenché par `?dexport=1`
`addrow(TableRow $row)`	une ligne de pied
`emptyStateRow()`	la ligne affichée quand la liste est vide

4.6 Le vocabulaire de la requête

Les paramètres que `Datatable` et `Lazyloading` lisent :

Paramètre	Lu par	Rôle
`dfilters=on`	`Datatable`	active le filtrage dynamique
`order=<colonne>`	les deux	la colonne de tri
`orderjoin=<relation>`	les deux	le tri sur une colonne jointe
`dsort` / `dsortjoin`	`Lazyloading`	variantes de tri
`next=<page>`	`Lazyloading`	la page
`per_page=<n>`	`Lazyloading`	la taille de page
`tablemodel=<X>`	`Datatable`	sélectionne `build<X>table()`
`dexport=1`	`Datatable`	déclenche l'export CSV
`dlang=<iso>`	`Lazyloading`	la langue
`drand`	`Lazyloading`	ordre aléatoire
`dcount` / `dsum`	`Lazyloading`	agrégats
en-tête `Columns`	`Lazyloading`	`with()` sans toucher au code

4.7 Les opérateurs de filtre

`Lazyloading::filterswicher()` (`Lazyloading.php:76`). La syntaxe est `?dfilters=on&<colonne>:<op>=<valeur>`.

Opérateur	SQL
`eq`	`=` (en AND)
`neq`	`!=`
`oreq`	`=` (en OR)
`gt` / `lt`	`>` / `<`
`get` / `let`	`>=` / `<=`
`lk`	`LIKE '%v%'`
`lkr`	`LIKE 'v%'`
`lkl`	`LIKE '%v'`
`in`	`IN (...)` — valeurs séparées par des virgules
`notin`	`NOT IN (...)`
`rein`	`<valeur> IN (<colonne>)` — sens inverse
`empty`	`= ''` si la valeur est `1`, `!= ''` sinon
`isNull`	`IS NULL` si `1`, `IS NOT NULL` sinon
`notNull`	`IS NOT NULL`
`btw`	`BETWEEN a AND b` — les deux bornes jointes par `_` : `created_at:btw=2026-07-01_2026-07-31`

> **`gte` et `lte` n'existent pas.** Un opérateur non reconnu tombe dans le `default` du `switch`, qui est un `LIKE`. `created_at:gte=X` génère donc `created_at LIKE '%X%'` : pas une comparaison, aucun avertissement, un résultat vide ou faux. **Toujours `get` / `let` / `btw` pour les comparaisons de nombres et de dates.**

Une colonne pointée est résolue comme une relation : `project.id:eq=1`. Et la syntaxe `<relation><alias>` avec un `<` demande une jointure supplémentaire (`extractXJoin()`).

Le filtrage dynamique ne combine qu'en ET. Pour un OU, il faut `whereGroup()` côté PHP (Tome II §4.2).

4.8 La réponse de `lazyloading`

Clé	Contenu
`success`	
`listEntity`	les entités de la page
`nb_element`	le total réel du filtre appliqué
`per_page`	
`current_page` / `next` / `previous`	
`remain`	`1` tant qu'il reste des pages, `0` sur la dernière

Clé	Contenu
`aggregations`	si `dcount` / `dsum` ont été demandés
`append_values`	ce que `append($key, $value)` a ajouté

Chapitre 5 — Le moteur de formulaires

5.1 Les trois classes

Classe	Rôle
`FormManager` (abstraite)	la classe de base d'un `<Entité>Form` ; définit le vocabulaire
`FormFactory`	**le moteur de rendu** : un `if/elseif` unique qui transforme une déclaration de champ en HTML
`Form` (extends `FormFactory`)	une API « à la main », pour écrire un formulaire hors du contrat déclaratif

5.2 Le contrat déclaratif

```
class TaskForm extends FormManager
{
    public static function init(\Task $task, $action = "")
    {
        return new TaskForm($task, $action);
    }

    public function buildForm()
    {
        $this->title = t('Tâche');
        $this->width = 700;

        $this->fields['name'] = [
            FH_TYPE  => FORMTYPE_TEXT,
            FH_LABEL => t('Nom'),
            FH_VALUE => $this->entity->name,
            FH_REQUIRE => true,
        ];

        $this->fields['description'] = [
            FH_TYPE  => FORMTYPE_TEXTAREA,
            FH_LABEL => t('Description'),
            FH_VALUE => $this->entity->description,
        ];

        $this->fields['project.id'] = [
            FH_TYPE    => FORMTYPE_SELECT,
            FH_LABEL   => t('Projet'),
            FH_VALUE   => $this->entity->project->getId(),
            FH_OPTIONS => FormManager::Options_Helper('name', Project::all()),
        ];

        return $this;
    }
}
```

Puis, côté contrôleur (`CrudTrait::formView()`) :

```
TaskForm::init($task, $action)->buildForm()->addDformjs()->renderForm();
```

5.3 Les types de champ

Constantes définies dans `FormManager.php:15-41`.

Constante	Valeur	Rendu
`FORMTYPE_TEXT`	`'text'`	`<input type="text">`
`FORMTYPE_EMAIL`	`'email'`	`<input type="email">`
`FORMTYPE_NUMBER`	`'number'`	`<input type="number">`
`FORMTYPE_PASSWORD`	`'password'`	`<input type="password">`
`FORMTYPE_DATE`	`'date'`	`<input type="date">`
`FORMTYPE_DATETIME`	`'datetime'`	`<input type="datetime-local">`
`FORMTYPE_TEXTAREA`	`'textarea'`	`<textarea>`
`FORMTYPE_SELECT`	`'select'`	`<select>`
`FORMTYPE_SELECTGROUP`	`'selectgroup'`	`<select>` avec `<optgroup>`
`FORMTYPE_CHECKBOX`	`'checkbox'`	une ou plusieurs cases
`FORMTYPE_RADIO`	`'radio'`	boutons radio
`FORMTYPE_FILE`	`'file'`	envoi d'un fichier
`FORMTYPE_FILEMULTIPLE`	`'filemultiple'`	envoi de plusieurs fichiers
`FORMTYPE_INJECTION`	`'injection'`	injecte un bloc HTML arbitraire
`FORMTYPE_INJECT_COLLECTIONFORM`	`'injectioncollection'`	**déclaré, non implémenté**

Et les sous-types de fichier : `FILETYPE_IMAGE`, `FILETYPE_DOCUMENT`, `FILETYPE_VIDEO`, `FILETYPE_AUDIO`.

> **`FORMTYPE_INJECT_COLLECTIONFORM` n'a aucune branche dans `FormFactory::__renderForm()`.** Un champ déclaré avec ce type tombe dans le `else` générique : rendu vide en production, un commentaire HTML en développement. Zéro occurrence dans tout `src/` de 3agReader. La constante est restée publique (un module hors dépôt pourrait s'y référer) et non implémentée (aucun exemple réel à suivre). **À ne pas utiliser.**
>
> **Aucun type n'est un répéteur.** Il n'existe pas de champ « liste de valeurs modifiable ». Sur 3agReader, un champ qui devait accueillir une liste de noms de paliers a été rendu en `FORMTYPE_TEXT` avec une consigne de séparation par virgules, et une validation dans le `setter`.

5.4 Les clés d'un champ

Constantes `FH_*` (`FormManager.php:43-53`) :

Constante	Valeur	Rôle
`FH_TYPE`	`'type'`	**obligatoire** : un `FORMTYPE_*`
`FH_LABEL`	`'label'`	le libellé
`FH_VALUE`	`'value'`	la valeur courante
`FH_OPTIONS`	`'options'`	les choix d'un `select`, `radio` ou `checkbox`
`FH_DIRECTIVE`	`'directive'`	les attributs HTML supplémentaires
`FH_ID`	`'id'`	l'identifiant HTML
`FH_FILETYPE`	`'filetype'`	`FILETYPE_*` pour un champ fichier
`FH_REQUIRE`	`'require'`	marque le champ obligatoire (ajoute l'astérisque)
`FH_IMBRICATE`	`'imbricate'`	formulaire imbriqué
`FH_COLLECTIONFORMINJECTION`	`'collectionforminjection'`	

`FormFactory` lit en plus, directement sur le tableau de champ : `values` (pluriel, pour les collections), `src` (l'URL d'un aperçu de fichier), `lang` (un champ traduit, rendu une fois par langue), `checker`, `placeholder`, `option`, `setter`, `hidden`, `callback`.

5.5 Les aides à la construction des options

Méthode	Rôle
`FormManager::Options_Helper($value, $entitylist, $key = "id")`	construit `[id => libellé]` à partir d'une liste d'entités ; `$value` peut être pointé (`'relation.name'`)
`FormManager::Options_ToCollect_Helper($value, $entity, $currentcollection)`	les options **moins** celles déjà sélectionnées
`FormManager::Options_ToCollectFormInjection_Helper(...)`	variante avec injection de formulaire
`FormManager::key_as_value(array $array)`	`[valeur => valeur]`, pour une énumération de chaînes

```
FH_OPTIONS => FormManager::Options_Helper('name', Project::all())
FH_OPTIONS => FormManager::key_as_value(['brouillon', 'publié', 'archivé'])
```

5.6 Les méthodes de `FormManager`

Méthode	Rôle
`buildForm()`	**à implémenter**
`addField($name, $option)`	équivalent de `$this->fields[$name] = $option`
`addJs($file)` / `addCss($file)`	ressources du formulaire
`addDformjs()`	charge `dform.js`, requis pour la soumission sans rechargement
`renderForm()`	rend le HTML via `FormFactory::__renderForm()`

Méthode	Rôle
`buildEntityCore()`	construit l'objet que `FormFactory` consomme
`printHtmlForm($entitycore, $rootdir)`	écrit le formulaire dans un fichier HTML

Propriétés : `$title`, `$width` (600 par défaut), `$entity`, `$classname`, `$name`, `$js`, `$css`, `$formbutton`, `$formaction`, `$fields`, `$viewdatas`.

5.7 L'API « à la main »

`Form` (extends `FormFactory`) permet d'écrire un formulaire champ par champ, dans une vue, hors du contrat déclaratif :

```
Form::open($entity, ["action" => "#", "method" => "post"]);
Form::input('name', $entity->name, ['class' => 'form-control']);
Form::textarea('description', $entity->description);
Form::select('project', $options, $entity->project->getId());
Form::file('cover', $entity->cover, [], FILETYPE_IMAGE);
Form::submit('Enregistrer');
Form::close();
```

Méthode	Rôle
`open($entity, $directives, $overideaction)` / `close()`	les balises de formulaire
`init($entity)`	
`imbricate($entity)` / `closeimbricate()`	formulaire imbriqué
`input($name, $value, $directive, $type, $setter)`	champ générique
`number()`, `email()`, `password()`	variantes typées
`textarea($name, $value, $directive, $setter)`	
`select($name, $options, $value, $directive, $setter, $callback)`	
`selectGroup($name, $options, $value, $callback, ...)`	
`autocomplete($name, $options, $value, ...)`	
`radio($name, $options, $value, ...)` / `checkbox($name, $options, $values, ...)`	
`file($name, $value, $directive, $filetype, $setter)`	
`filepreview($value, $src, $directive, $filetype)`	aperçu
`inputarray($name, $key, $value, ...)`	champ de tableau
`submit()` / `submitbtn()` / `reset()`	boutons
`addDevupsjs()`, `addJquery()`, `addDformjs()`, `addjs($js)`, `addcss($css)`	ressources

5.8 Un champ traduit

Si l'entité est traduite et que le champ porte `'lang' => true`, `FormFactory` rend **un champ par langue active**, alimenté par `FormFactory::$langs` (`Dvups_lang::allrows()`, positionné par `renderForm()`).

> **Attention, lié au défaut D15** (Tome II §7.4) : une fiche d'édition d'une entité traduite **sans aucune ligne de traduction** se charge vide, parce que l'entité est invisible à l'ORM. Le formulaire ne l'explique pas : il affiche des champs vides. Un garde a été posé à l'enregistrement (`Model::__update()` refuse avec un motif clair) ; l'affichage, lui, reste muet.

Chapitre 6 — Les vues

6.1 Blade, via `philo/laravel-blade`

Les vues sont des gabarits Blade (`*.blade.php`), compilés dans `cache/views`. Les chemins de recherche sont dans le tableau global `$viewdir`, enrichi à l'exécution :

Contexte	Chemins
application publique	`web/views`, `admin/views`
écran d'entité	plus `src/<composant>/<Module>/Resource/views` (posé par `Controller::views()`)
écran de module	plus `src/<composant>/<Module>/Resource/views` (posé par `Controller::viewsModule()`)

6.2 `Genesis` — le rendu

Méthode	Rôle
`Genesis::render($view, $data)`	rend une vue avec `web/views` et `admin/views` comme racines
`Genesis::renderView($view, $data, $redirect)`	rend une vue avec `$viewdir` global ; ajoute `moduledata` aux données
`Genesis::getView($view, $data)`	rend une vue **et retourne la chaîne** au lieu de l'émettre
`Genesis::renderBladeView($view, $data, $action, $redirect)`	variante historique
`Genesis::json_encode($value, $options, $depth)`	**émet du JSON et termine la requête** (`die`) ; pose `Content-Type` et un en-tête `X-Exec-Time`
`Genesis::json_encode2(...)`	la même chose **sans** `die`
`Genesis::top_action($action, $statefull)`	construit la barre d'actions du haut d'une liste, en tenant compte des droits de l'administrateur

> **`Genesis::json_encode()` termine la requête.** C'est volontaire et c'est une règle : tout ce qui doit s'exécuter après l'émission de la réponse doit l'être avant l'appel. Les deux `die` ajoutés dans `WebService::systemServe()` sur 3agReader (voir chapitre 7) existent précisément parce qu'un `json_encode()` de refus **sans** `die` laissait l'exécution continuer et l'écriture se faire quand même.

6.3 La directive `@tt`

`BladeD::registerBladeEngine()` enregistre une directive personnalisée :

```
@tt('clef')
@tt('clef', 'valeur par défaut')
```

qui compile vers `<?php echo tt($ref, $default); ?>`.

6.4 Les fonctions de traduction

Fonction	Rôle
`t($clef, $defaut = "")`	la traduction d'administration
`tt($clef, $defaut = "")`	la traduction du site public
`langt($fr, $en, $local, $matcher)`	deux chaînes en dur, choisies par la langue ; `$matcher` remplace des `:jetons`
`local()`	la langue courante : `?lang=`, puis la session, puis `__lang`
`setlang($lang)`	fixe la langue en session

6.5 Les fonctions globales utiles

Déclarées dans `dclass/dvups_functions.php` :

Fonction	Rôle
`route($path, $id, $title)` / `d_url(...)`	construit une URL avec le préfixe de langue (sauf en administration)
`path($src)`	`__env . $src`
`d_assets($src)`	`__env . 'web/assets/' . $src`
`services($service)`	`__env . 'admin/api/' . $service`
`redirect($url)`	`header('location: ...')` puis `die`
`getadmin()`	l'administrateur en session, désérialisé ; une instance vide sinon
`setadmin($id)`	met un administrateur en session
`url_format($str)` / `remove_accents($str)` / `clean($str)`	assainissement de chaînes
`get_string_between($string, $start, $end)`	
`qb_sanitize($value)`	**entoure une chaîne de quotes sans l'échapper** — voir Tome II §8
`dv_dump(...$args)`	`dump()` puis `die(1)` — **un arrêt, pas un journal**

6.6 Le JavaScript du socle

`dclass/devupsjs/`, servi par la constante `CLASSJS` :

Fichier	Rôle
`devups.js`	le socle
`model.js`	le modèle côté client : `model._new()`, `model._edit()`, la soumission
`ddatatable.js` + `DdatatableClass.js`	la liste : rechargement, tri, filtres, pagination

Fichier	Rôle
`dform.js`	la soumission d'un formulaire sans rechargement
`dialogbox.js`	les fenêtres modales
`Request.js`, `rest.js`	les appels HTTP
`sha1.js`	

Une liste de back-office fonctionnelle demande `devups.js`, `model.js` et `ddatatable.js` ; un formulaire demande en plus `dform.js` — c'est ce que fait `addDformjs()`.

Chapitre 7 — La sécurité

7.1 Les deux systèmes d'identité

Devups a **deux** mécanismes d'authentification, indépendants, qui ne se connaissent pas.

	API publique	Back-office
Support	jeton JWT	session PHP
Porté par	en-tête `Authorization: Bearer <jwt>` ou `Auth: Bearer <jwt>`	`$_SESSION[ADMIN]`
Identité	`Auth::$user_id` (un `User`)	`getadmin()` (un `Dvups_admin`)
Vérifié par	`Auth::execute()`, en tête de `runServe()`	`Router::manage()`

Conséquence structurelle : dans une requête d'administration, `Auth::$user_id` est **toujours vide**. Tout contrôle écrit en termes d'`Auth::$user_id` bloquerait donc l'intégralité du back-office. C'est pourquoi `Router::$admin_route` existe (§7.5).

7.2 Le JWT

`Auth::getJWT(\User $user)` émet le jeton ; `Auth::execute()` le vérifie.

Élément	Valeur
Algorithme	HS512
Secret	`jwt_secret_Key`
Charge utile	`iss` (= `__server`), `userId`, plus `iat`/`nbf`/`exp` **si `jwt_expire_time` est vrai**
Expiration	contrôlée **seulement** si `jwt_expire_time` est vrai

```
$jwt = Auth::getJWT($user);                  // à la connexion
$token = Auth::decodeJWTOrFail($jwt);         // forme stricte : lève sur tout échec
$token = Auth::decodeJWT($jwt);                // forme historique : renvoie un tableau
d'échec
```

> **Deux points à connaître.**
>
> `jwt_expire_time = false` sur 3agReader : les jetons n'expirent jamais, et aucune vérification d'`iss`/`nbf`/`exp` n'a lieu. Seule la signature est vérifiée.
>
> `decodeJWT()` **retourne silencieusement un tableau d'échec** qu'un appelant peut oublier de tester. `decodeJWTOrFail()` a été ajoutée pour cela : elle lève sur tout jeton absent, mal formé, invalide ou expiré. **À utiliser pour tout code neuf** qui décode un jeton hors du flux par annotation.

7.3 `@Auth`, et où il est vérifié

Les trois dispatchers ne traitent pas `@Auth` de la même façon :

Dispatcher	Traitement
`Router::mapFunction()`	si la méthode porte `@Auth`, exige `Auth::$user_id` non vide, sinon exception
`Request::FrontController()`	si la méthode porte `@Auth`, appelle `$Auth->execute()` et lève sur échec
`FrontController::authorization($classname, $method)`	consulte `Auth::$restrictions` (§7.4)

> **Important.** Dans `Router::mapFunction()`, `@Auth` contrôle que `Auth::$user_id` est peuplé — ce qui a déjà été fait par `Auth::execute()` en tête de `runServe()`. Il n'y a pas de double décodage. En revanche, l'absence de `@Auth` sur une méthode `xxxServe()` signifie **aucun contrôle** : la route est anonyme.

7.4 `Auth::$restrictions` — le modèle réel

C'est le mécanisme central, et il fonctionne par **liste d'exceptions**, pas par liste d'autorisations.

```
// route/WebService.php
Auth::addRestriction($entity, self::$publicMethods[$entity] ?? []);
```

Lu ainsi (`Auth.php:24`) : pour l'entité `$entity`, les méthodes de `$methodException` sont **publiques** ; **toutes les autres exigent un jeton**.

Et le contrôle (`Request.php:555`) :

```
if (isset(Auth::$restrictions[$dclass])
    && !in_array($function, Auth::$restrictions[$dclass])
    && !Auth::$user_id) {
    http_response_code(401);
    throw new Exception("Bad Request AUTHORIZATION or AUTH not found");
}
```

Toute entité enregistrée dans `$public_access` et absente de `$publicMethods` est donc protégée sur toutes ses méthodes — les cinq actions génériques comme ses actions métier. C'est le bon sens de la liste : c'est elle qui doit rester courte et relisible, pas une exception à documenter ailleurs en prose.

Le nom comparé est celui de la méthode **réellement résolue** par le routeur : `index` pour `GET /api/<entité>`, `detail` pour `GET /api/<entité>/<id>`, le nom de l'action pour une route annotée.

> **Un défaut corrigé, instructif.** Ce contrôle était placé **plus haut** dans la méthode, avant le bloc de résolution par annotation qui réaffecte `$function`. Sur une route de sous-ressource, il comparait donc l'**ancienne** valeur — le chemin par défaut du verbe, `index` pour un GET — et non la méthode qui allait être appelée. La correction (déplacer le contrôle après la résolution) est préventive : aucune route réelle ne déclenchait la branche au moment de la mesure.
>
> **Le défaut réellement exploitable était ailleurs, et il est corrigé** : le style générique **exécutait l'écriture malgré un refus d'autorisation**, faute d'un `die` après le `json_encode()` du refus. Vérifié en direct : `POST /api/create/comicbook` sans jeton émettait bien le refus, puis **poursuivait sa route** jusqu'à `createCore()`. Voir §7.6.

Comment construire `$publicMethods`

Le principe appliqué sur 3agReader, parce qu'il est transférable :

> La lecture anonyme est le cœur du produit — on lit avant de s'inscrire — et le client mobile n'envoie le jeton **que s'il en a un**. Toute lecture qu'un visiteur non connecté peut atteindre doit donc figurer dans `$publicMethods`, sinon elle tombe en `401`. Inversement, rien de ce qui touche au compte, à l'écriture ou à l'argent n'y figure.

La liste a été établie par **recensement croisé** de tous les chemins d'API réellement appelés par les clients, confrontés aux méthodes réellement déclarées dans chaque `FrontController`. Les actions laissées protégées sont **nommées en commentaire** sous chaque entité, pour qu'on ne les prenne pas pour des oublis :

```
// /comicbook, /comicbook/:id, /comicbook/ranking, /comicbook/chapters,
// /comicbook/comments, /comicbook/stats, /comicbook/completeData.
// Protégées : subscribe (écrit un abonnement), publish, create, update, delete.
'comicbook' => ['index', 'detail', 'ranking', 'chapters', 'comments',
                'stats', 'completeData'],
```

C'est la forme à reprendre : **la liste porte sa propre justification.**

7.5 Le contrôle de propriété

`Auth::$restrictions` exige un jeton, mais **pas que la ligne appartienne au porteur du jeton**. Un second mécanisme s'en charge, dans `FrontController` :

```
private static $ownerColumn = [
    'comicbook'   => 'user_id',
    'chapter'     => 'user_id',
    'post'        => 'user_id',
    'postcomment' => 'user_id',
    'illustration'=> 'user_id',
    'follow'      => 'follower_id',
];

private function assertOwnership($id, $classname)
{
    if (Router::$admin_route || !isset(self::$ownerColumn[$classname]) || !$id) {
        return;
    }
    $column = self::$ownerColumn[$classname];
    $newclass = self::$classMagic;
    $owned = $newclass::where(['this.id' => $id,
                               'this.' . $column => Auth::$user_id])->count();
    if (!$owned) {
        http_response_code(403);
        throw new Exception("Vous n'etes pas autorise a modifier cette ressource");
    }
}
```

Appelé en tête de `updateCore()`, `deleteCore()` et `uploadCore()`.

Trois décisions de conception à retenir :

1. L'absence d'une entité de la liste est un choix, pas un oubli. Une entité absente reste protégée par le jeton — toute écriture exige une session — mais n'a pas de contrôle de propriété supplémentaire. Les entités concernées sont nommées en commentaire (`member`, `slide`, `contest`, `participant`, `product`, `payment`, `orders`, `donate`, `analytic`, `dvups_entity`).

2. Le contrôle passe par une requête dédiée, pas par la propriété de l'entité hydratée. `$entity->user_id` **n'est pas fiable partout** : `Post`, entre autres, n'expose pas `user_id` comme propriété PHP (seulement la relation `$user`), et `ModelTrait::__get()` meurt sur un attribut non déclaré. Un `$entity->user_id` aurait donc fait planter `deleteCore()` au lieu de refuser proprement, pour cette entité précise. Le `where()->count()` est agnostique de la forme d'hydratation de chaque entité.

3. `Router::$admin_route` est le garde-fou indispensable. Ces trois méthodes sont **partagées** avec `AdminService::systemServe()`, qui sert le back-office derrière sa propre session, et n'appelle jamais `FrontController::authorization()`. `Auth::$user_id` y est donc toujours vide. **Sans le `if (Router::$admin_route) return;`, ce contrôle aurait bloqué toute écriture d'administration sur ces six entités** — alors qu'un back-office ne doit évidemment pas être restreint à « ses propres » lignes.

7.6 Le garde-fou des entités d'argent

`WebService::systemServe()` porte un second refus, après l'autorisation :

```
if (
    in_array($classname, self::$genericPathMoneyEntities, true)
    || (
        in_array($classname, self::$genericPathWriteBlockedEntities, true)
        && in_array(self::$path, self::$genericPathWriteActions, true)
    )
) {
    http_response_code(403);
    g::json_encode([
        'success' => false,
        // Ne dit pas si la ligne existe : ce chemin ne sert ni lecture ni
        // écriture pour cette entité, quel que soit l'id.
        'detail' => 'this path does not serve this entity',
    ]);
    die;
}
```

Raison d'être : `authorization()` exige bien un jeton pour ces entités, mais **aucune propriété**. Une entité d'argent ne doit pas être joignable par le chemin générique du tout.

Noter deux choses dans ce bloc, qui sont des règles générales :

- **le `die`** — sans lui, le refus est encodé en JSON et **l'exécution continue** vers le `switch` qui exécuterait l'écriture. Ce défaut exact a existé et a été vérifié en direct ;
- **le message ne révèle rien** — il ne dit pas si la ligne existe, et il est identique quel que soit l'id.

7.7 CORS

`runServe()` compare l'origine à `__csrf_origin`. Un en-tête `Access-Control-Allow-Origin` n'est émis que si l'origine y figure ou vaut `null` (auquel cas `*` est renvoyé).

Les en-têtes autorisés sont codés en dur dans `Router.php:364` : `Authorization, Auth, X-Customer-Auth, Columns, Dlang, common, Content-Type, X-Requested-With`. **Un en-tête personnalisé nouveau doit être ajouté là**, sinon le navigateur refuse la requête au vol préliminaire.

7.8 Un cas de code mort qui ressemble à de la sécurité

`Router::authorization()` a été **supprimée** de `Router.php`. Elle n'avait aucun appelant, et elle utilisait l'ancienne signature positionnelle de `firebase/php-jwt` 5.x (`JWT::decode($jwt, $secret, $hashArray)`), incompatible avec la 6.x installée : tout appel aurait levé une `TypeError` immédiate.

Le commentaire laissé en place dit le principe, et il vaut d'être repris :

> « Du code mort qui ressemble à de la sécurité est pire que pas de code du tout. »

7.9 Liste de vérification

Point	À vérifier
nouvelle entité exposée	est-elle dans `$public_access` ? dans `$publicMethods` **à bon escient** ?

Point	À vérifier
nouvelle action d'écriture	`@Auth(authorized=1)` posée ? le nom de la méthode est-il **absent** de `$publicMethods` ?
écriture sur une ressource appartenant à un utilisateur	l'entité est-elle dans `$ownerColumn` ? sinon, est-ce un choix documenté ?
nouveau refus dans un flux générique	y a-t-il un `die` après le `json_encode()` ?
message de refus	révèle-t-il l'existence de la ressource ?
nouvel en-tête HTTP	ajouté à `Access-Control-Allow-Headers` ?
nouvelle méthode de `FrontController`	porte-t-elle une annotation de verbe ? sinon elle est **morte**
décodage d'un jeton hors annotation	`decodeJWTOrFail()`, pas `decodeJWT()`

Fin du Tome III. Le Tome IV traite des modules fournis par le socle, du protocole de cache des données de référence, et des dettes connues du framework.